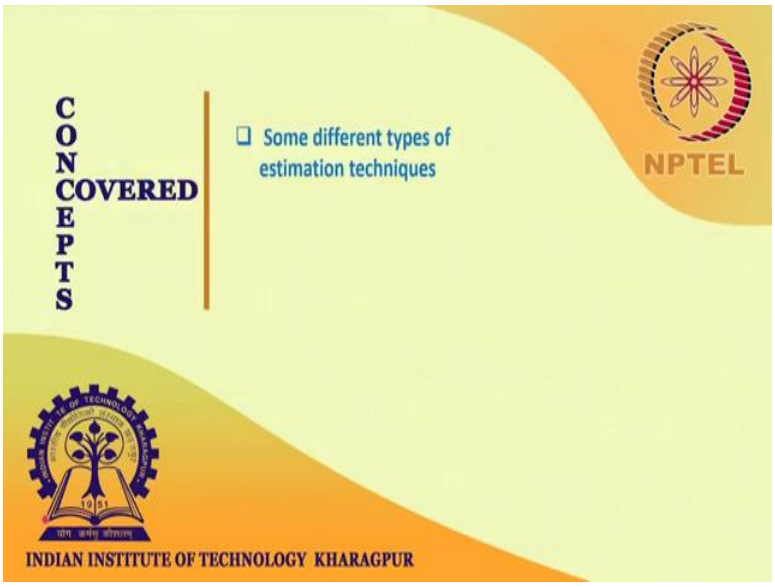
Software Project Management
Prof. Durga Prasad Mohapatra
Department of Computer Science and Engineering
National Institute of Technology, Rourkela

Lecture – 18
Project Estimation Techniques (Contd.)

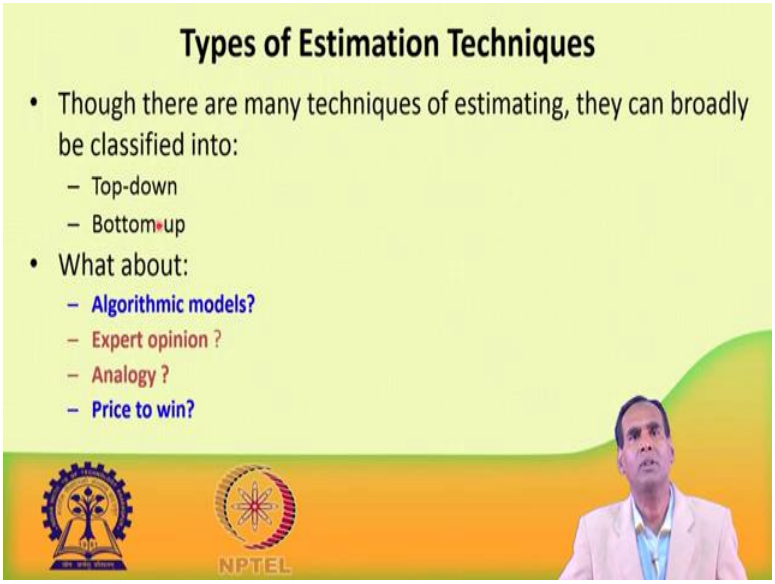
Good afternoon to all of you, we will take up some other categories of estimation techniques now ok.

(Refer Slide Time: 00:26)



We will see some different other types of estimation techniques.

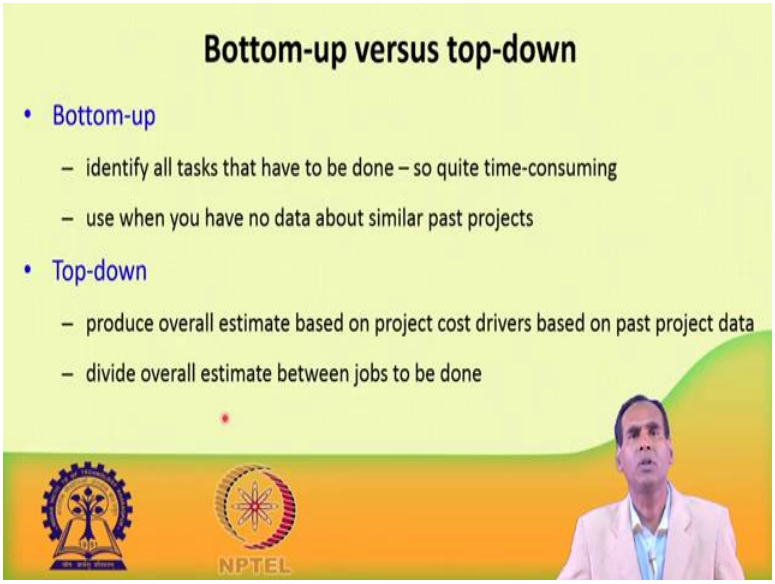
(Refer Slide Time: 00:31)



The types of, there is another category of different types of estimation techniques though, there are many techniques of estimation they can be broadly classified into top down and the bottom up, this is one classification.

So, what about: algorithm models, Expert opinion, Analogy, Price to win? In the last class, we have discussed about the these last three expert opinion or expert judgment, analogy mythology and price to win, those things already we have seen and so, basically these four are also another classification for project estimation. So, the last three already we have seen. Now, let us see about the top down and the bottom up estimation techniques then we will see about the algorithmic models for project estimation.

(Refer Slide Time: 01:18)

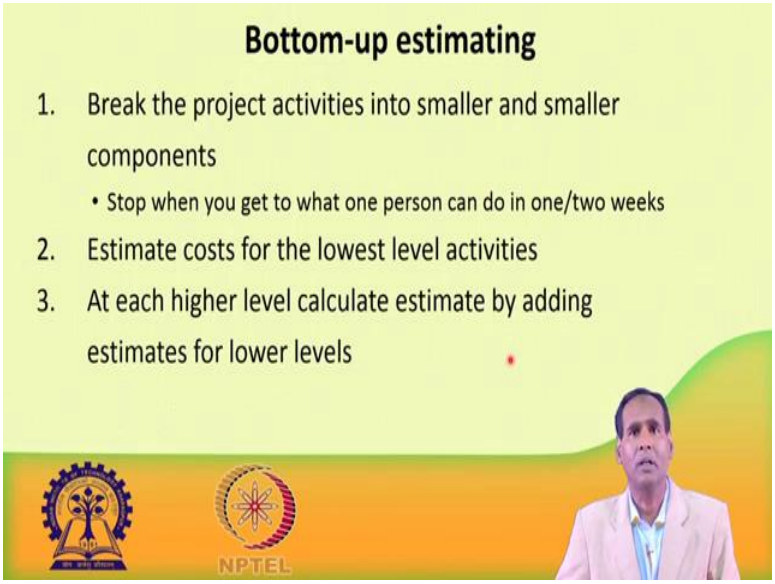


So, bottom up estimation is a estimation technique which identifies all tasks that have to be done ok. So, so it identifies all the tasks that have to be performed. So, it is very much time consuming. Use when you have no data about similar past projects. So, when you do not have similar past projects, you may use this see I have already told you that analogy method.

Analogy method is based on when it will be suitable when you are having similar kinds of past projects or similar kinds of projects you have done in the past then this analogy method you can use, but when you do not have any data about the similar past projects, then you use this bottom up approach. Then what is about top down approach? It produce overall estimates based on project cost drivers and based on the past project data.

So, top down method is used when this past project data are available. So, this method produces overall estimates based on two things; the project various project cost drivers based on what the past project data. So, here we have to divide the overall estimate between the jobs they have to be performed.

(Refer Slide Time: 02:39)

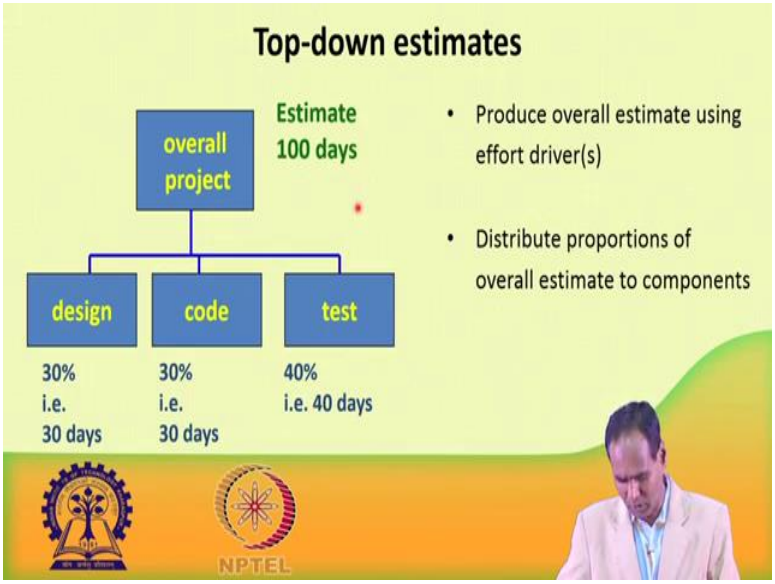


Now, let us see first bottom up estimating or bottom up estimation. What are the steps? The steps for bottom up estimation are as follows; break the project activities into smaller and smaller components. So, take as its name suggest bottom up estimation, just like in object oriented approach you are using bottom up what mythodology.

So, here similarly you break the project activities into smaller and smaller components, then stop when you get to what to be done by one person in one or two weeks then. Where to this breaking process? When it should be stop? We should stop when you get that what the piece of work that one person can do within one or two weeks, then you estimate the cost for the lowest level activities.

So, after breaking the projects into activities, smaller activities then you estimate the cost for the lowest level activities at each higher level. At each higher level then you calculate the estimates how; by adding the estimates for lower levels. So, at each higher level, you have to calculate the estimates by summing up the estimates for the lower levels.

(Refer Slide Time: 03:55)



Let us then let us see about the top down estimates, we will take an example. So, here suppose, as top down estimate I have already told you earlier, I think I have told that ok. Let us see bottom up approach, bottom up estimation algorithm, we have already seen. So, now, let us see about this top down estimation, difference already I have told you. Here, the top down estimates was as follows like produce an overall estimate using produce, the overall estimate using the effort drivers or the cost drivers you have to use here some drivers to produce the overall estimate and then you have to distribute the proportions of the overall estimate to various components.

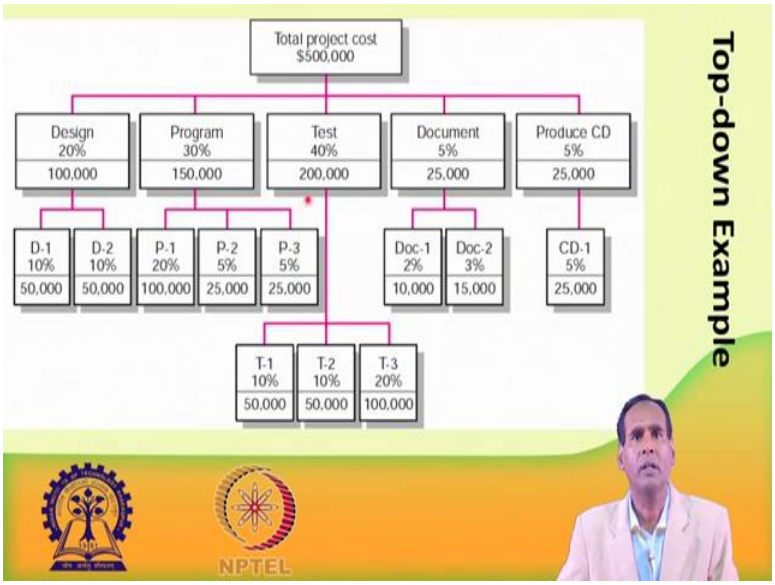
Here, you can see that for the overall project you have estimated that you it might take 100 days, but the overall project can be divided into what different what sub activities like; design, coding, testing, etcetera and you have estimated that 30 percent, 30 percent of the total days can be given to design, 30 percent to code, and 40 percent you will give to test.

Then you can easily calculate the proportionate the proportionate of this what total estimate like 30 percent of 100 days that is 30 days for design, again 30 percent of the total estimate 100 days, again it is 30 days for coding and 40 percent of the total estimated that is 40 days for the testing.

So, in this way the top down estimate works, first estimate the overall estimate, first calculate the overall estimate then distribute the proportions of the overall estimate to the

individual components then you will. Then this method of estimating the effort or the duration is known as a top down estimation.

(Refer Slide Time: 05:45)



Another example is given here, where these the total. Suppose, the total project cost is 500000 then the total project, overall project total project what are the different components in developing the total project like design, coding, testing, documenting, produce the CDs, etcetera.

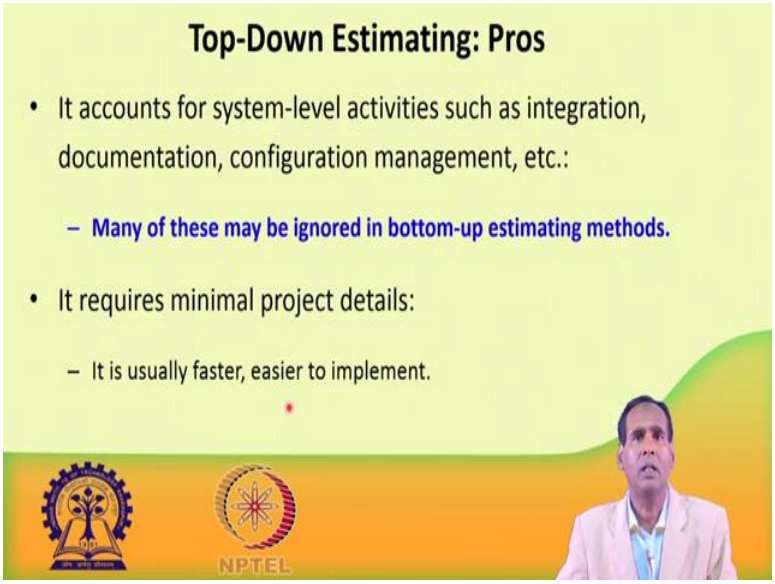
So, then what you have to do, the person you have to estimate how much percentage will be taken up by each component; suppose, design will take 20 percent, programming will take 30 percent, testing will take 40 percent, documentation will take 5 percent and producing CD will take 5 percentage. Accordingly, this you have to compute, this say 20 percent of 500000 that is 100000, it will give to what, it will give to the design. Similarly, 30 percent of 500000 maybe 1.5 lakhs, it will give to programming and similarly, you assign this what calculate the proportions and calculate the percentages assign to different components.

So, similarly design can be say two types of design D 1, D 2 like; D 1 design and your database design the percentage at 10, 10 ,10 percentage each. So, you have to give 50000 each.

So, similarly program there can be two programming shows write writing down the code for the G U I and for the say may be here three programming's; say programming for this G U I, programming for the database and programming for the normal code. So, again 20 percent, 5 percent, 5 percent. So, the proportionate are calculated, documentation suppose, two types CD, prepares only one.

So, in this way you can divide this, what proportionate similarly, testing a divide into unit testing is T 1 integration testing T 2 system testing might be T 3. So, accordingly compute this percentage you will get the estimates for each of the phases. So, this is how the top down estimation this works.

(Refer Slide Time: 07:40)

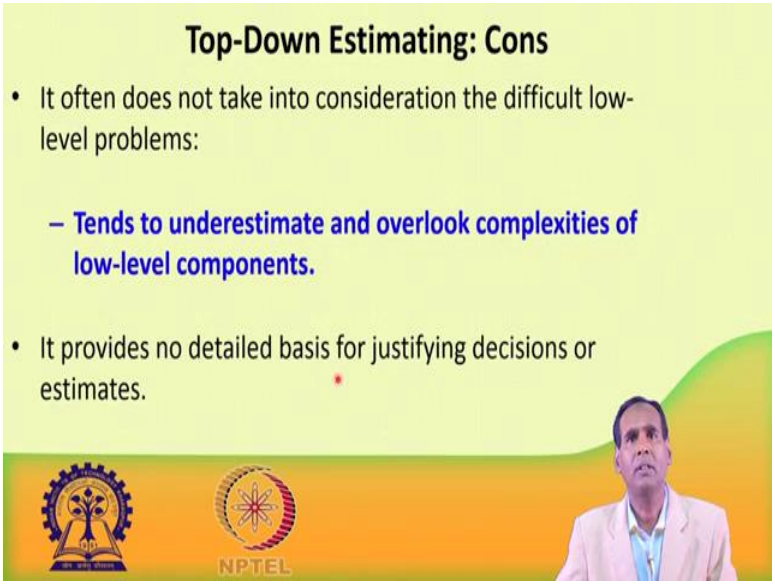


Let us see what are the advantages of top down approach, it accounts for system level activities such as integration testing or the integration, documentation, configuration management, etcetera.

So, top down estimation, it considers the various system level activities such as integration, documentation, configuration management, etcetera. Many of these may be ignored in bottom up estimation methods. We will see one of the drawback of bottom up estimation is that it does not take into account to this these what system level activities.

It requires minimal project details. So, this top down estimation, it requires very few amount of project details, very minimal project details. It is usually faster and easier to implement. These are the advantages of top down estimation.

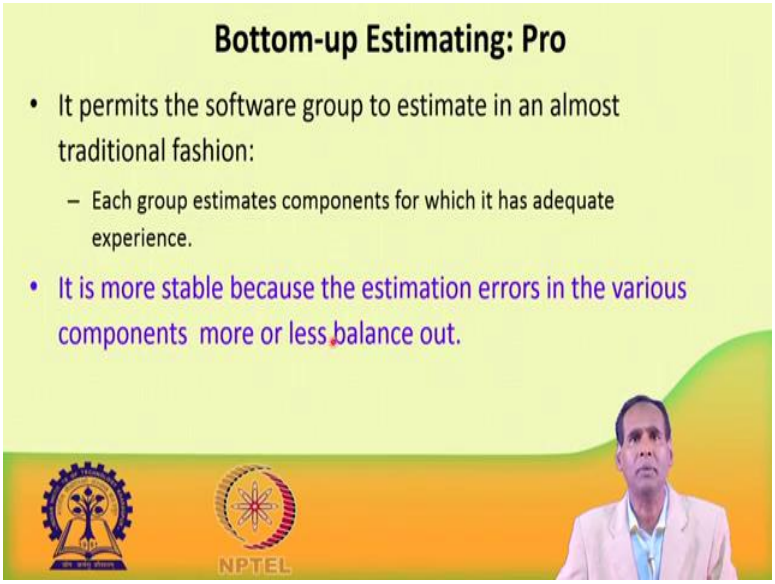
(Refer Slide Time: 08:30)



But there are also some disadvantages of the top down estimation such as; it upon does not taken to consideration the difficult low level problems, it takes to account this what system level activities, but it does not take into account the low level of problems. It tends to underestimate and overlook the complexity of the low level components. It does not take into account as well as it may tend to underestimate and overlook the complexities of low level components. It provides no details no detailed basis for justifying the decisions or estimates.

So, why you have taken this decision? Why you have done this estimate? It does not provide any detailed basis for justifying the decisions or the estimates that a project manager has made..

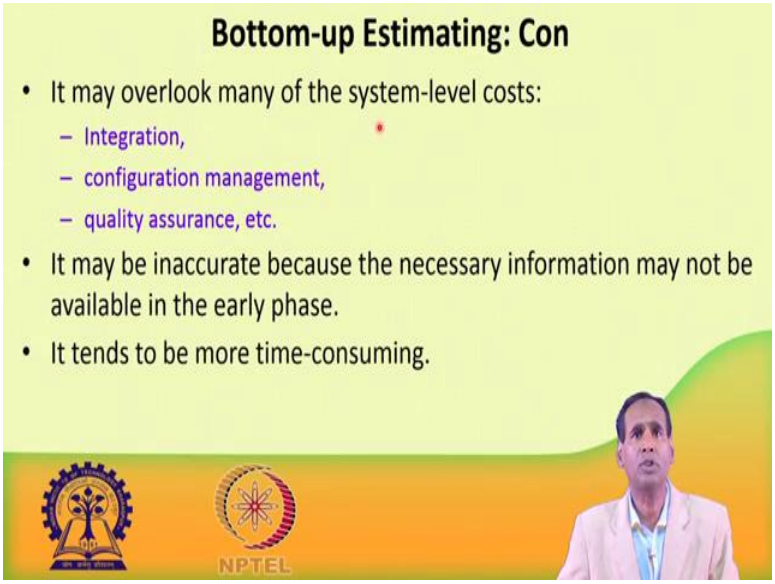
(Refer Slide Time: 09:18)



The on the what are let us see some of the what the advantages of bottom up estimation. It permits the software group to estimate in an almost traditional fashion. So, in a traditional fashion the software group or the project manager, they can estimate the various parameters. Each group estimate components for which is has adequate experience.

So, different groups involved in the software development, they will estimate different components for which they have adequate experience. It is more stable, because the estimation errors in the various components more or less balance out. So, different it is it is very stable, because the estimation errors in the various components, they are already balanced.

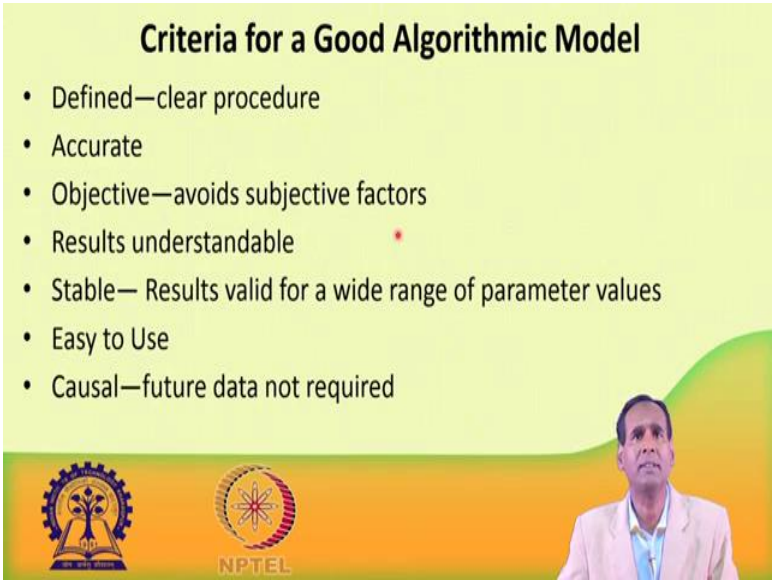
(Refer Slide Time: 10:04)



But the drawback, there are several drawbacks of bottom up estimation such as it may overlook many of the system level cost. This I have already told you earlier, the advantages of top down approach is that it takes into account the system level activities, but bottom up estimation may overlook many of the system level costs such as integration, configuration management, etcetera, quality assurance.

So, the cost that will be involved in this system level cost may be overlooked by this bottom up estimation process. It may be inaccurate, because the necessary information may not be available in the early phase. So, during the early phase may be like a system requirement system requirement analysis phase or this design phase. So, the necessary information may not be available. So, the estimate may be inaccurate if you are using this bottom up estimation. It is tends to be more time consuming, it is more it is more time consuming in comparison to the bottom up estimation.

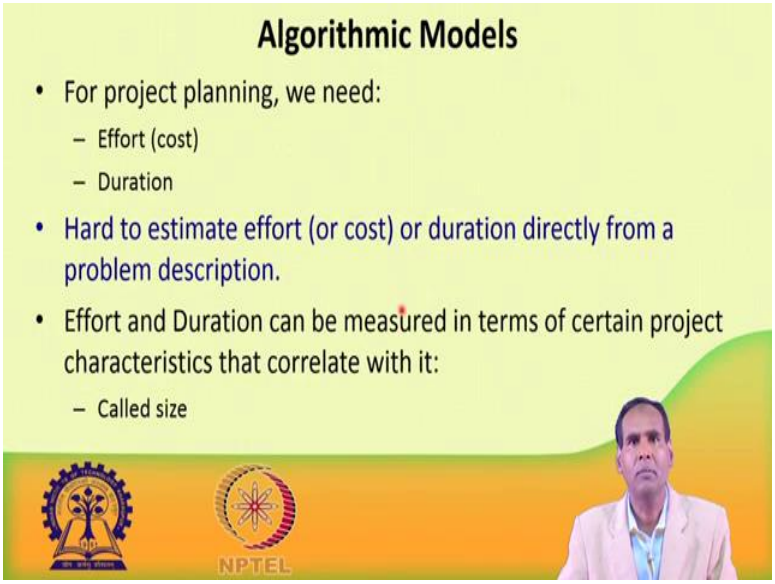
(Refer Slide Time: 11:04)



So, now let us see about another model called as the algorithmic model for this project estimation. Now, first let us see what are the criteria required for a good algorithmic model, then we will take up for the steps for this algorithmic model.

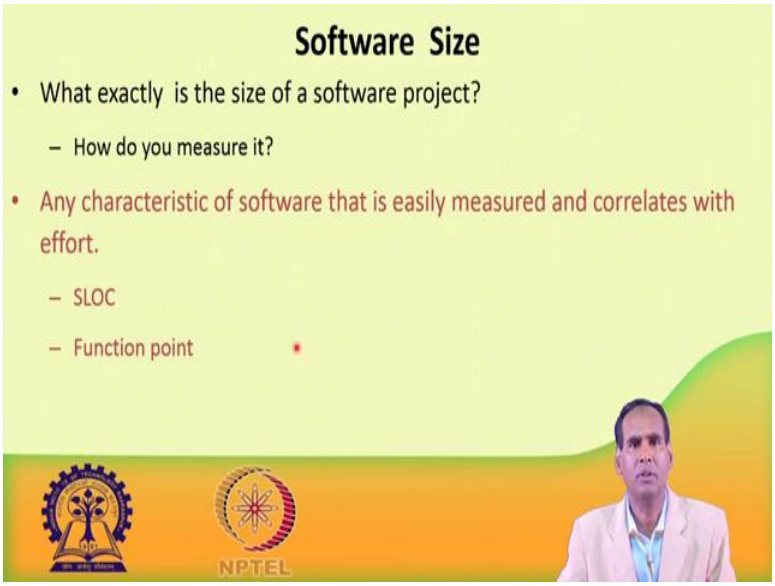
So, first of all this algorithmic model, if you want to develop a algorithmic model for project estimation, it should be clearly defined, the procedure should be very clearly defined. Then it is must be accurate. It should be objective; that means, you should avoid the subjective factors. It should only concentrate on the objective factors. The results should be understandable and it should be stable; that means, the results must be valid for a wide range of parameter values, not for a small range of parameter values and it should be easy to use. It should be very much causal; that means, future data not required only with the existing data current data. We should be able to estimate the various parameters. These are some of the criteria for a good algorithmic model.

(Refer Slide Time: 12:09)



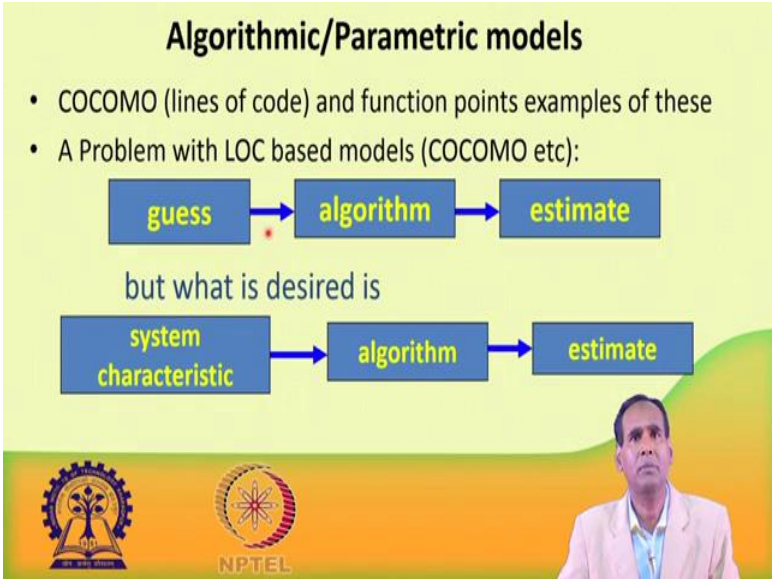
Now, let us see what could be the algorithm models. For project planning, we need to estimate what the report as well as the duration. We have already told you that directly estimating the effort duration this is very much difficult it is hard to estimate effort or cost or duration directly from a program description. So, that is why effort and the duration they can be measured in terms of certain project characteristics that correlate with it, for example, it is that call size, by using the size we can estimate effort and duration. So, first we have to estimate the size, by using the size estimate we can then estimate the effort and duration.

(Refer Slide Time: 12:52)



So, software size we have already discussed earlier, that what exactly this is this size of a software project, how do you measure it and I have already told you, we may use that two parameters S L O C and function point. So, any characteristic of software that is easily measured and correlates with effort that decides,, any characteristic of software that is easily measured and correlates with effort is known as what size and how can you measure it using what the metrics such as S L O C and function point or F P.

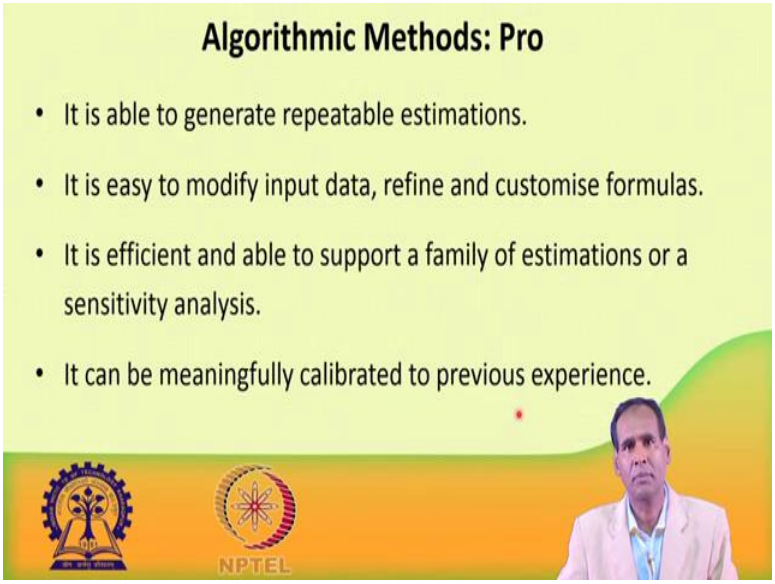
(Refer Slide Time: 13:24)



Now, this let us see the algorithmic models or now, let us see this algorithmic models or parametric models. So, two important examples are there; one is COCOMO of this lines of code and the function points, they are examples of these models.

Let us see one of the model with the L O C based model or the COCOMO. In either the L O C based model or the COCOMO we initially guess some value, we initially guess some value then apply the algorithm and estimate, but what actually desire is, we should what estimate the values not from starting from the guess, we should estimate the values starting from starting from the system characteristics, then we can apply the algorithm and then we can estimate. So, this is the problem with L O C based models or the COCOMOS. So, in order to overcome this, we may apply algorithmic or parametric models.

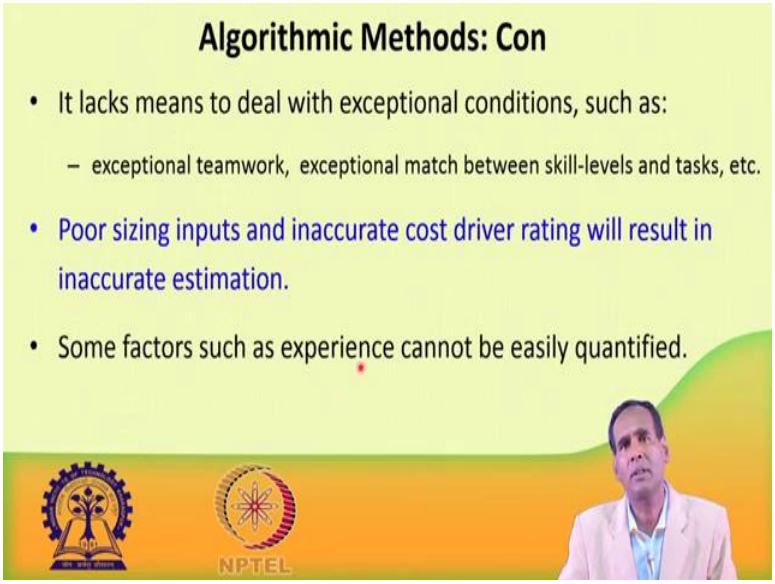
(Refer Slide Time: 14:23)



Now, let us see what are the advantages of algorithmic methods. It is able to generate a repeatable estimations. So, once you estimate for some parameter, you can generate repeatable estimations.

So, that is why it is repeatable in nature. It is the estimation that you are obtaining using algorithmic methods they are easy to modify ok. They are easy to modify input data, refine and customize the different formula. It is efficient and able to support a family of estimations or a sensitive analysis. So, this algorithm method it is efficient and it is able to support not only single estimations, but a family of estimations or a sensitive analysis. It can be meaningfully calibrated to previous experience. So, one of the most important advantage is that, it can be very meaningfully calibrated, you can do the calibration to previous experience.

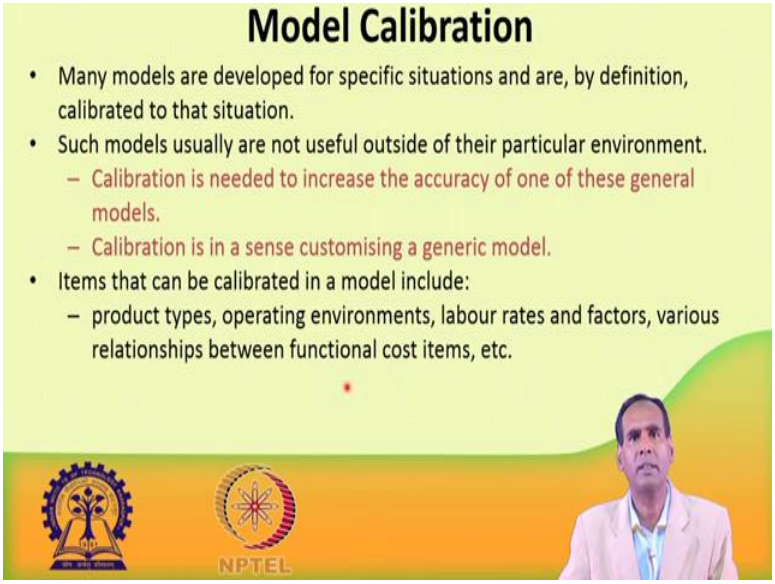
(Refer Slide Time: 15:23)



There are some drawbacks of algorithmic methods such as a it lacks the means to detail with to deal with exceptional conditions. So, a algorithm methods, they can deal with some exceptional conditions such as exceptional teamwork, exceptional match between skill levels and task etcetera. So, these exceptional conditions can easily sorry, they cannot be what dealt with using this algorithm methods.

So, poor sizing inputs and inaccurate cost driver rating will result in inaccurate estimation. So, if you are having what poor sizing inputs, if the cost driver ratings are inaccurate, then ultimately, they will result in inaccurate estimation. Some factors such as experience cannot be easily quantified. So, if you are using algorithmic methods. So, factors such as experience of somewhat coder or experience of some tester, it cannot be easily quantified.

(Refer Slide Time: 16:25)



So, now let us as I have already told you that here one of the drawback sorry, one of the advantage here, I have already told you that the algorithmic methods, they can be meaningfully calibrated to previous experience. Let us see how we can do calibration to the previous experiences using the algorithmic methods. So, let us see the calibration, model calibration.

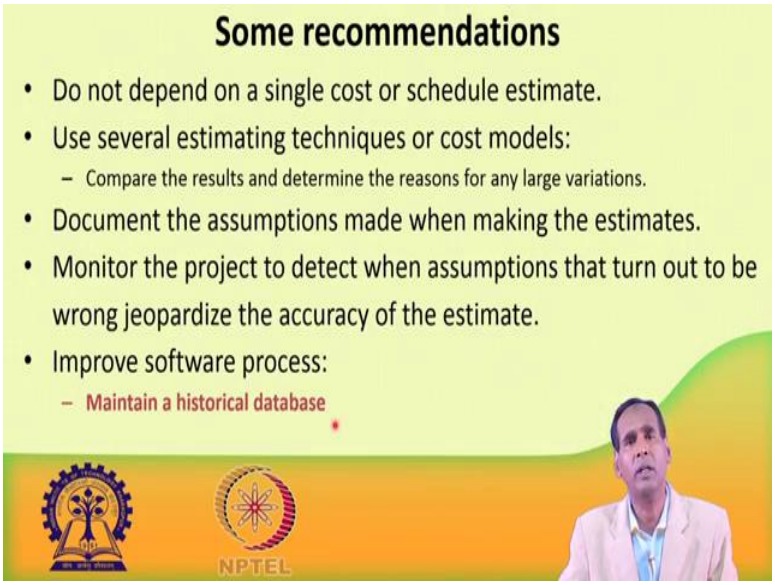
So, many models are developed for specific situations and are by definition calibrated to that situation. So, there are several models, they are developed for only specific situations, they are not generalized, they are developed for specific situations and by definition, they can be calibrated to that situation. Such models usually are not useful outside of that particular environment.

I have, as I have already told you, these models will be very much what a suitable for the specific situation, they cannot be useful outside the particular environment. Calibration is needed to increase the accuracy of one of these general models. So, why calibration is required? Calibration is needed to increase the accuracy of one of these general models. Calibration is a sense customizing a generic model. We have to a generic model is there and we want to customize it according to our own needs.

So, calibration is in a sense customizing a generic model. Items that can be calibrated in a model include let us say in a model what items can be calibrated. You can calibrate you can calibrate the product types, we can calibrate the operating environments. Similarly,

the other items they can be calibrated are labour rates and factors, various relationships between the functional cost items, etcetera.

(Refer Slide Time: 18:19)



Now, let us see some of the recommendations for this what calibration. Do not depend or let us see some of the recommendations while using this estimation models some of the recommendations while using or while doing estimation. Do not depend on a single cost or schedule estimate. We have already seen that various methodologies are there. So far we have seen bottom up, top down and this what algorithmic, then this judgment based, pricing so many, so do not depend on a single cost or schedule estimate, because sometimes if you are using a single estimate or single technique, it may give wrong result.

So, that is why you may use several estimating technique or cost models, instead of using just a single estimation technique or a single model, you may use different or several estimating techniques or cost models, so that the result may be very much accurate. Then if you are using different estimation techniques or models for estimation, then what you can do after applying the different techniques or models for finding out or for estimating the parameters, then you may compare the results and then determine the reasons for any large variations.

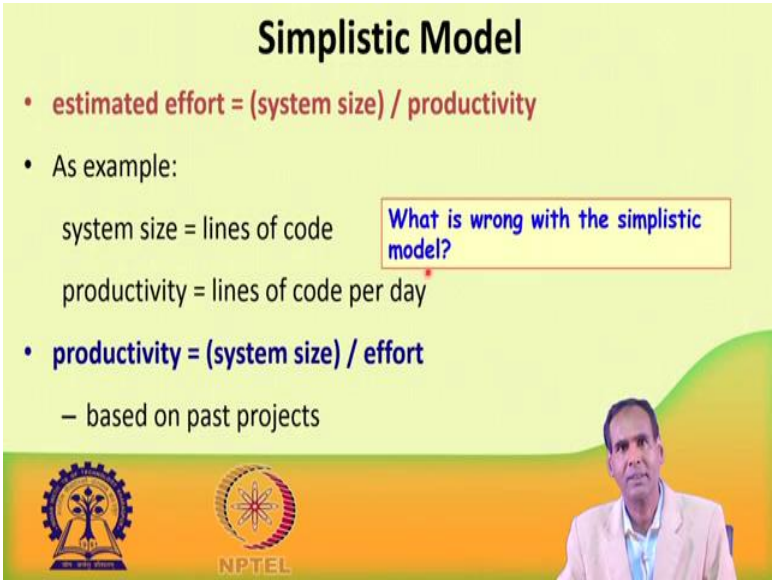
They should; so whatever method may use, they should give very closer results. If the results are in the results there are large variations they may, you may analyze, why this

large variations are coming. So, then you note down them then document the assumptions made when making the estimate.

So, while you are using different techniques or models for estimation you may use different assumptions. You may take different assumptions, but whatever assumptions you have taken so you have to document all the assumptions met, when making the estimates. Then monitor the project to detect when assumptions that turned out to be wrong jeopardize the accuracy of the estimate. So, sometimes due to wrong assumptions, your accuracy, your estimate it becoming what less accurate, it is becoming wrong. So, monitor the project to detect when the assumptions that will turn out to be wrong jeopardizing the accuracy of the estimate, then improve the software process ok.

So, analyzing the estimates and if you are what observing that due to this assumptions you are getting what, you are getting wrong estimates, then you may improve this software process. So, how again; you can also improve the software process the whole process by maintaining a historical database. So because sometimes we require the past data, so you have to improve the software process by maintaining the historical database, containing the details of the past projects. So, these are some of the recommendations that you may follow as a project manager while carrying out project estimations.

(Refer Slide Time: 21:34)



Now, let us take this the concept of simplistic model. According to this a very simple model, according to this model the effort is estimated as follows; estimated the effort is equal to system size divided by productivity and very straightforward formula.

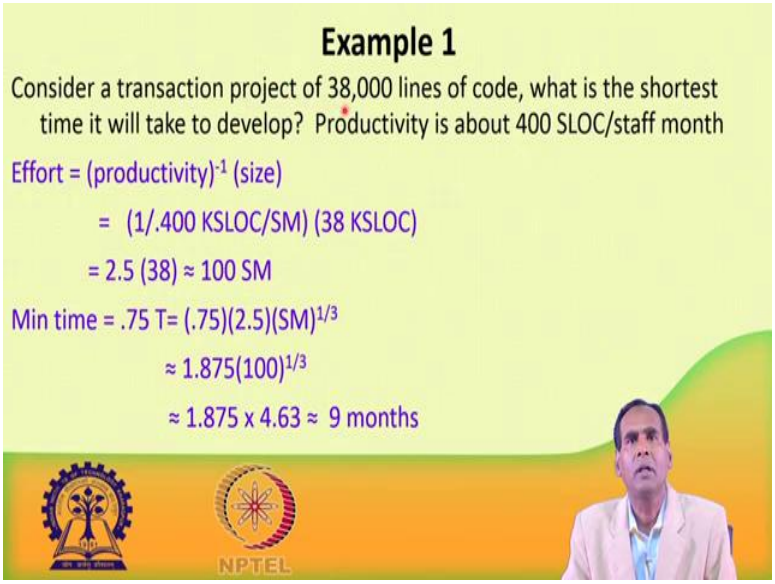
$$\text{estimated effort} = (\text{system size}) / \text{productivity}$$

So, if the system size is some lines of code and the productivity is some lines of code per day, then productivity from this equation, you can find out that productivity equal to how much? System size divided by effort.

$$\text{productivity} = (\text{system size}) / \text{effort}$$

So, here this formula is based on the data relating to the past projects, but please think of what is wrong with the simplistic model.

(Refer Slide Time: 22:26)



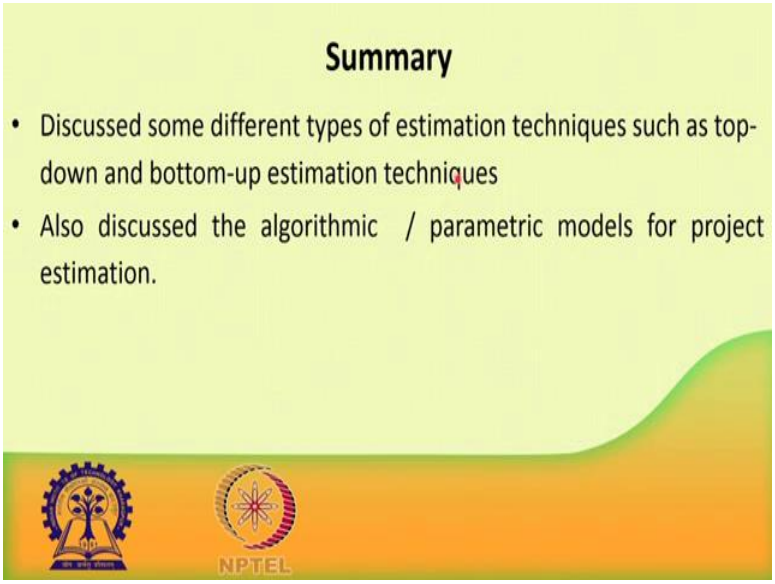
So, in which cases it will fail? So, please think of on this. Now, let us quickly take a small example; we are, we want to find out what these effort and the effort and this duration, we want to estimate the effort and this duration for developing this project. This example says that consider a transaction project of 38000 lines of code. So, this transaction project contains 38000 lines of code, we have to found out what is the shortest time that it will take to develop this project. Here, it is also given that productivity is about 4000 S L O C per staff month.

So, the productivity is given in terms of 400 S L O C per staff month. Now, we have to do this. As we can see that the formula of effort I have already given you earlier. So, effort can be calculated as system size by productivity or in that you can say that effort is equal to productivity to the power minus 1 or 1 by productivity into size, this is equal to 1 by productivity is given as 400 L O C or K L O C divided by ok, 1 by 0.400 K S L O C per staff month into size. Size is equal to how much? Size is equal to 38 equal 38000 lines of code; that means, 38 K S L O C, K S L O C. So, this is coming to be something this 2.5 into 38 is equal to approximately, 100 staff months will be required for developing the project.

So, here the estimated effort is coming to be 100 staff month. Now, the minimum time is required is equal to we have to find out that. So, this will be equal to 0.75 T and which is equal to 0.75 and T here, it is this effort 0.7 and into 2.5 that we have already got this thing, 1 by 400 something. So, this is equal to 0.75 into 2.5 S M to the power 1 by Q and this is coming to be 1.875 into and simplification it is coming to be around 9 months. So, for this transaction project having 38000 lines or 38 K S L O C and productivity of 400 S L O C for staff month we get that that, the effort is estimated to be 100 staff months and minimum time is estimated to be 9 months.

So, similar examples are there, you can get from the book or in places. Please, try to solve them, also some exercises you may solve them. So, these types of small exercises may be asked in the assignments as well as in the examination.

(Refer Slide Time: 25:40)



So, finally, we have discussed some different types of so, let us see the summary we have discussed some different types of estimation techniques here, in this class such as top down testing we have seen and bottom up estimation techniques we have seen and we have also compare the top down, the top down approaches and the bottom up estimation techniques. Also we have taken some examples to illustrate the top down and the bottom up estimation techniques. Then as also we have also discussed the algorithmic model or the parametric model for project estimation.

We have seen the advantages and disadvantages of algorithmic or parametric models then we have seen one simplistic model for project estimation. We have also taken an example for estimating the effort and time by using and this simplistic model for project estimation. So, these are another classifications of techniques for project estimation. In the next class we will discuss about another metric for estimating size that is function point that we will discuss in the next class. So, these are the references and we will stop here.

(Refer Slide Time: 16:50)



Next class, we will see the remaining parts of this project estimation techniques.

Thank you very much.